Elemente de DB Temporale in Produse Comerciale

Introducere

- Consideram urmatoarele: o tabela *Orders* cu definitia

```
Create table Orders(
        order_number int not null primary key,
        product_id int not null,
        qty   int not null,
        order_date date
);
```

Am dori sa aflam perioadele dintre comenzi pentru fiecare produs

- Cum am scrie o astfel de interogare?

Introducere

ORDER_NUMBER	PRODUCT_ID	QTY	ORDER_DATE
1	1000	10	10-MAY-16
2	3000	11	02-JUN-16
3	1000	20	23-JUN-16
4	2000	33	11-SEP-16
5	3000	3	12-NOV-16
6	1000	12	13-APR-17

```
SELECT product_id, order_date,
LAG (order_date,1) OVER (PARTITION BY product_id
ORDER BY order_date) AS prev_order_date
FROM orders order by 1, 2;
```

PRODUCT_ID	ORDER_DATE	PREV_ORDER_DATE
1000	10-MAY-16	(null)
1000	23-JUN-16	10-MAY-16
1000	13-APR-17	23-JUN-16
2000	11-SEP-16	(null)
3000	02-JUN-16	(null)
3000	12-NOV-16	02-JUN-16

Introducere

- Bazele de date relationale moderne, cum ar fi **Oracle**, **Microsoft SQL Server** si **PostgreSQL**, ofera facilitati avansate de gestionare a datelor temporale. Acestea sunt esentiale pentru a pastra si gestiona versiuni multiple ale datelor in timp, permitand interogarea starilor anterioare si urmarirea modificarilor

Introducere

- Valid Time poate fi relevant in scenarii de:
  - **Information Lifecycle Management** (ILM) si alte aplicatii unde este important sa se cunoasca momentul cand anumite date au devenit valide (din perspectiva aplicatiei) si cand a devenit nevalida (daca a devenit)
  - **Data correction** cand trebuie stocate date incorecte si marcata perioada cand au fost considerate valide, perioada cand datele corectate sunt vazute ca fiind valide

Oracle VT si TT

- **Valid Time** în Oracle este gestionat printr-o funcționalitate denumită **Temporal Validity**. Aceasta permite adăugarea și definirea explicită a perioadelor de valabilitate pentru înregistrările din tabele. Utilizatorii pot defini perioade **PERIOD FOR** care indică timpul valid în care datele sunt considerate active sau relevante. Aceasta oferă suport direct pentru valid time, permițând interogarea și gestionarea datelor în funcție de valabilitatea lor într-o anumită perioadă de timp.

Oracle VT si TT

- **Transaction Time** este gestionat implicit de Oracle prin funcționalitatea **Flashback Data Archive** (FDA). Aceasta păstrează toate modificările datelor în tabelele arhivate, inclusiv timpul exact în care tranzacțiile au fost realizate. Prin intermediul funcțiilor de interogare AS OF și VERSIONS BETWEEN, utilizatorii pot reveni la stările anterioare ale datelor și pot accesa istoricul complet al modificărilor, ceea ce reprezintă o gestionare automată a transaction time.

Tehnologia Flashback

- Este folosita cand apare o corupere logica in baza de date Oracle si este necesara o recuperare rapida si usoara. Cand au loc erori umane este greu de identificat ce obiecte au fost afectate de o tranzactie eronata.

Tehnologia Flashback permite sa:

- diagnosticam erorile introduse in baza de date si sa reparam defectele.
- vizualizam tranzactiile ce au contribuit la modificarea unei inregistrari
- vizualizam toate versiunile unei inregistrari dintr-o anumita perioada de timp
- vizualizam datele cum existau la un anumit moment in timp in trecut.

Flashback – Facilitati DBA

- **Flashback Database**: restaureaza intreaga baza de date la un anumit moment in timp, folosind loguri optimizate pentru Oracle flashback, (alternativa la backup si forward recovery).
- **Flashback Table**: restaurarea facila a unei tabele la un anumit moment in timp; utila cand coruperea logica afecteaza o tabela sau o multime restransa de tabele din db.
- **Flashback Drop**: restaureaza o tabela stearsa in mod accidental. Restaureaza atat tabela cat si indexurile, constringerile si triggerii folosind Recycle Bin (un container logic pentru toate obiectele sterse).
- **Flashback Transaction**: realizeaza o anulare (undo) a efectului unei singure tranzactii, si optional, a tuturor tranzactiilor dependente, folosind o singura operatie PL/SQL sau un wizard din Enterprise Manager.

Flashback – Facilitati App Dev

- **Flashback Transaction Query**: vizualizare tuturor modificarilor unei anumite tranzactii; util cand o tranzatie eronata modifica mai multe inregistrari intr-o tabela.
- **Flashback Query**: interogheaza date la un moment specific in trecut. Se poate utiliza pentru a observa si a reconstrui date corupte logic, ce au putut fi modificate sau sterse necorespunzator.
- **Flashback Versions Query**: furnizeaza diferite versiuni ale unei inregistrari in cadrul unei perioade de timp.

Tehnologia Flashback

- Flashback Database foloseste **fisiere log flashback** pentru operatia de flashback. Flashback Drop foloseste **recycle bin**. Restul tehnicilor folosesc **undo data**.
- Nu toate capabilitatile flashback modifica baza de date. Unele sunt doar metode de interogare a versiunilor datelor. Acestea reprezinta unelte de analiza a problemelor si ajuta la recuperarea la erori. Rezultatele acestor interogari ajuta la:
  - Determinarea tipului de operatie (ce modifica db-ul) flashback pentru rezolvarea problemei .
  - Furnizeaza rezultatul interogarii unei instructiuni INSERT, UPDATE, sau DELETE ce permite repararea datelor eronate.

Beneficiile Flashback

- Tehnologia Flashback este un avans revolutionar in recuparea la erori
- Tehnicile traditionale de recuperare sunt lente
  - intreaga baza de date sau un fisier trebuie restaurat, nu doar datele incorecte
  - fiecare schimbare trebuie examinata in fisierele de log

- Flashback este rapid
  - Schimbarile sunt indexate dupa inregistrare si dupa tranzactie
  - Doar datele modificate sunt restaurate

- Comenzile Flashback sunt simple
  - Proceduri ce nu presupun pasi complexi multipli

Tehnologia Flashback

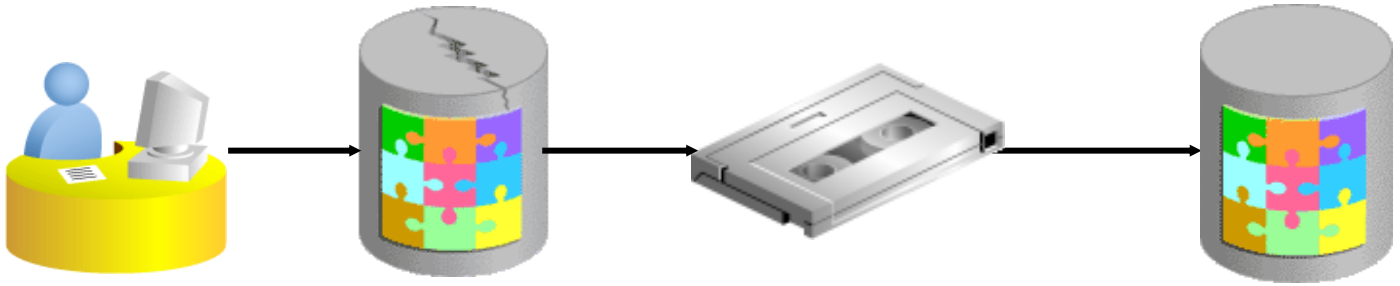
Nivelul Obiect	Scenarii exemplu	Tehnologia Flashback	Utilizare	Date afectate
Database	Truncate table; Modificari nedorite in mai multe tabele	Database	Flashback logs	TRUE
Table	Drop table	Drop	Recycle bin	TRUE
	Update cu clauza `WHERE` gresita	Table	Undo data	TRUE
	Compara date curente cu date din trecut	Query	Undo data	FALSE
	Compara versiuni ale unei inregistrari	Version	Undo data	FALSE
Tx	Investigheaza stari anterioare ale datelor	Transaction	Undo data	FALSE

Flashback Database

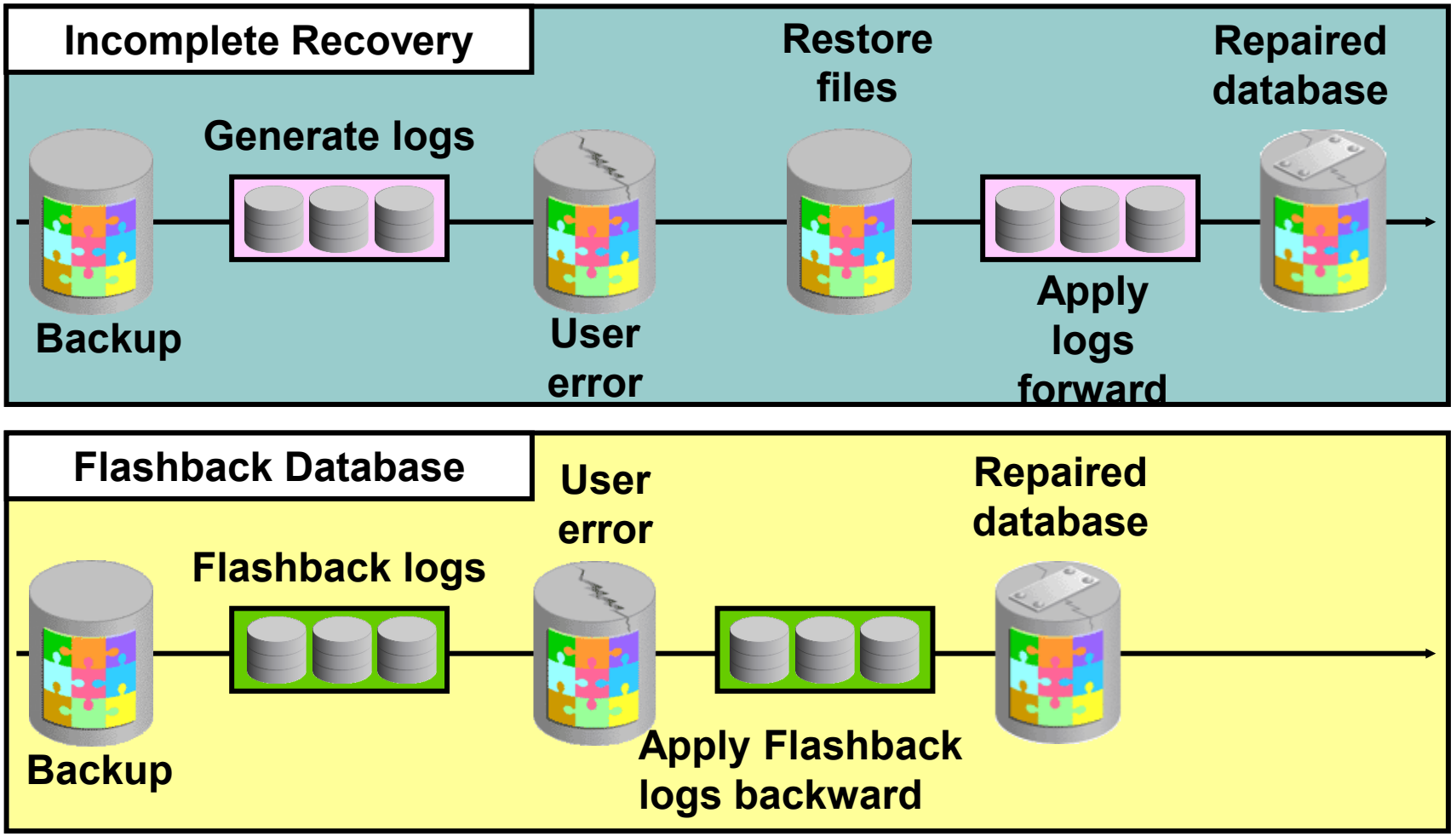
- Restaurarea facila si rapida a bazei de date la un moment anterior in timp prin anularea tuturor modificarilor ce au avut loc de la acel moment. Operatia este rapida pentru ca nu se restaureaza un backup. Se poate utiliza pentru anularea modificarilor ce au dus la coruperea logica a datelor.
- Pentru pierderi ale mediului fizic sau corupere fizica, se pot folosi doar metode traditionale de recuperare

Flashback Database

- Operatia Flashback Database:
  - Functioneaza ca un buton rewind pentru baza de date
  - Se poate folosi in cazuri de corupere logica facuta de utilizator



Flashback Database
Reduce Timpul de Restore



Configurare Flashback Database



1. Configure the flash recovery area
2. Set the retention target
3. Enable Flashback Database

```
SQL>  ALTER SYSTEM SET
            DB_FLASHBACK_RETENTION_TARGET=2880
            SCOPE=BOTH;
```

```
SQL> ALTER DATABASE FLASHBACK ON;
```

Monitorizare Flashback Database

- Ajustare dimensiune flash recovery disk quota:

```
SQL> SELECT estimated_flashback_size,
            flashback_size
     FROM   V$FLASHBACK_DATABASE_LOG;
```

- Determinare fereastra flashback curenta:

```
SQL> SELECT oldest_flashback_scn,
     oldest_flashback_time
     FROM   V$FLASHBACK_DATABASE_LOG;
```

- Monitorizare logging in Flashback Database logs:

```
SQL> SELECT *
    FROM   V$FLASHBACK_DATABASE_STAT;
```

Exemple Flashback Database

```
RMAN> FLASHBACK DATABASE TO TIME =
    > TO_DATE('2019-05-27 16:00:00',
    > 'YYYY-MM-DD HH24:MI:SS');

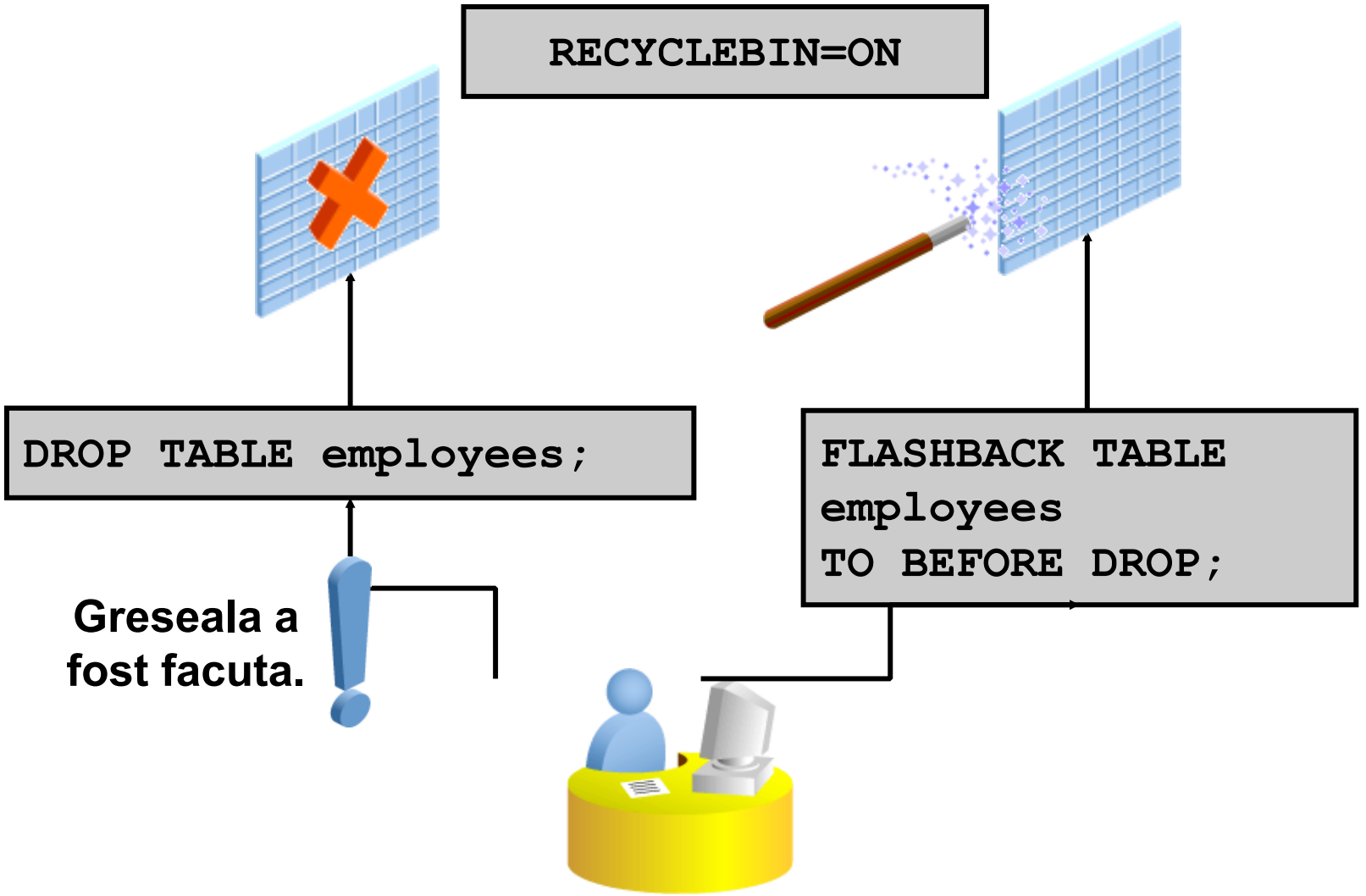
RMAN> FLASHBACK DATABASE TO SCN=23565;

RMAN> FLASHBACK DATABASE
    > TO SEQUENCE=223 THREAD=1;
```

```
SQL> FLASHBACK DATABASE
     TO TIMESTAMP(SYSDATE-1/24);

SQL> FLASHBACK DATABASE TO SCN 53943;
```

Flashback Drop si Rec. Bin



Flashback Drop

- In versiunile mai vechi ale Oracle database, daca se stergea o tabela din greseala, trebuia restaurata baza de date la un moment anterior pentru a recupera tabela. Procedura este de obicei de durata si resulta in pierderea efectelor unor tranzactii.
- In Oracle Database 10g se introduce facilitatea Flashback Drop, ce se poate utiliza pentru a anula efectul unei instructiuni DROP TABLE fara a folosi “point-in-time recovery”.
- Parametrul de initializare RECYCLEBIN este folosit pentru a configura existenta/inexistenta capabilitatii Flashback Drop prin valoarea ON, respectiv OFF. Daca este OFF, tabela stearsa nu va fi plasata in recycle bin. Daca este ON, tabela stearsa va fi plasata in recycle bin si poate fi restaurata. Implicit, parametrul RECYCLEBIN are valoare ON.

Flashback Drop

- Comanda FLASHBACK TABLE ... TO BEFORE DROP este utila pentru a restaura o tabela si toate obiectele dependente din recycle bin. Se poate specifica pentru restaurare numele original sau cel atribuit de sistem avut cand obiectul fost sters.
- Daca se specifica numele original si in recycle bin exista mai multe obiecte cu acelasi nume, obiectul cel mai recent mutat in recycle bin este restaurat primul (LIFO). Pentru o versiune anterioara se poate specifica numele generat de sistem sau se executa instructiuni FLASHBACK TABLE ... TO BEFORE DROP pana cand tabela dorita este restaurata.

Flashback Drop

- Daca in schema destinatie exista deja un obiect cu numele tabelei sterse, la restaurare se va genera eroare daca nu este specificata clauza RENAME TO.
- La operatia de flashback a unei tabele sterse, elementele atasate recuperate, indexuri, triggeri, constraingeri, vor avea numele din recycle bin. Se recomanda ca pentru obtinerea unor nume mai semnificative sa se interogheze recycle bin si DBA_CONSTRAINTS inainte de flashback pentru a redenumi aceste obiecte atasate cu nume sugestive.

Interogare Recycle Bin

```
SELECT owner, original_name, object_name,
       type, ts_name, droptime, related, space
FROM dba_recyclebin
WHERE can_undrop = 'YES';
```

```
SELECT original_name, object_name,
       type, ts_name, droptime, related, space
FROM user_recyclebin
WHERE can_undrop = 'YES';
```

```
SQL> SHOW RECYCLEBIN
```

Flashback Query

- Pentru Oracle Flashback Query se foloseste o instructiune SELECT cu o clauza **AS OF**. Oracle Flashback Query returneaza datele existente la un anumit moment in trecut. Interogarea refera in mod explicit momentul de timp cu ajutorul unui timestamp sau a unui System Change Number (SCN). Va furniza valorile ce au fost salvate (committed) la acel moment de timp.
- Utilizarea Oracle Flashback Query include:
  - Recuperarea datelor sterse sau anularea (undo) schimbarilor incorecte salvate (commited): de exemplu, din greseala s-au actualizat sau sters randuri, apoi s-a salvat, se poate imediat anula acest lucru.
  - Compararea datelor curente cu corespondentul lor din trecut.

Flashback Query

- Exemplu: se poate rula un raport zilnic ce arata toate schimbarile fata de ziua anterioara. Se pot compara inregistrari individuale ale tabelei, sau, se pot determina intersectii, reuniuni de inregistrari.
- Simplifica arhitectura aplicatiei eliminand nevoia de stocare a unor atribute temporale: Oracle Flashback Query permite furnizarea unor date din trecut direct din baza de date.
- Permite un mecanism self-service de corectie a erorilor, permitand utilizatorilor sa anuleze sau sa corecteze erorile.

Flashback Query

```
CREATE TABLE my_emp(
  empno NUMBER,
  last_name VARCHAR2(30),
  start_time TIMESTAMP,
  end_time TIMESTAMP,
PERIOD FOR user_valid_time (start_time,
end_time));

INSERT INTO my_emp VALUES (100, 'Ames', '01-Jan-
10', '30-Jun-11');
INSERT INTO my_emp VALUES (101, 'Burton', '01-
Jan-11', '30-Jun-11');
INSERT INTO my_emp VALUES (102, 'Chen', '01-Jan-
12', null);
```

Flashback Version Query

- Utilizat pentru regasirea versiunilor unei anumite inregistrari ce a existat intr-un interval de timp. O noua versiune a inregistrarii este creata atunci cand este implicata intr-o operatie de COMMIT.
- Oracle Flashback Version Query se specifica prin clauza VERSIONS BETWEEN a unei instructiuni SELECT. Sintaxa este:

```
VERSIONS {BETWEEN {SCN|TIMESTAMP} start AND end}
```

Unde start si end sunt expresii ce reprezinta inceputul si sfarsitul intervalului de interes. Intervalul include (start si end).

- Oracle Flashback Version Query furnizeaza o tabela cu o inregistrare pentru fiecare versiune a randului ce a existat pe durata intervalului specificat. Fiecare inregistrare va contine pseudocoloane (metadata) despre versiunea inregistrarii.

Flashback Version Query

VERSIONS_STARTSCN
VERSIONS_STARTTIME

System Change Number-ul (SCN) de inceput sau TIMESTAMP-ul cand a fost creata versiunea inregistrarii. Aceasta pseudocoloana indica momentul cand inregistrarea a avut valorile reflectate in versiune. Pseudocoloana se poate folosi pentru a determina timpul tinta pentru Oracle Flashback Table sau Query. Daca pseudocoloana este NULL, versiunea a fost creata inainte de timpul de start.

VERSIONS_ENDSCN
VERSIONS_ENDTIME

SCN sau TIMESTAMP-ul cand versiunea inregistrarii a expirat. Daca este NULL, fie versiunea corespunde unui DELETE, fie versiunea era curenta la momentul interogat.

Flashback Version Query

VERSIONS_XID

Identificatorul tranzactiei ce a creat versiunea inregistrarii.

VERSIONS_OPERATION

Operatia efectuata in tranzactie: **I** - insert, **D** - delete, **U** - update.
Versiunea corespunde inregistrarii dupa un INSERT, inainte de DELETE, respectiv UPDATE.
Pentru actualizarea valorii unei chei de index Oracle Flashback Version Query poate trata UPDATE ca fiind 2 operatii, DELETE plus INSERT, fiind prezente 2 versiuni, prima cu **D** uramat de **I** in VERSIONS_OPERATION.

Flashback Version Query

```
SELECT versions_startscn, versions_starttime,
       versions_endscn, versions_endtime,
       versions_xid, versions_operation,
       name, salary
  FROM employees
  VERSIONS BETWEEN TIMESTAMP
      TO_TIMESTAMP('2003-07-18 14:00:00', 'YYYY-
MM-DD HH24:MI:SS')
  AND TO_TIMESTAMP('2003-07-18 17:00:00', 'YYYY-
MM-DD HH24:MI:SS')
  WHERE name = 'JOE';
```

Flashback Transaction Query

- Utilizat pentru regasirea metadatelor si istoricului pentru o anumita tranzactie sau pentru toate tranzactiile dintr-un interval de timp. Oracle Flashback Transaction Query interogheaza view-ul dictionarului de date FLASHBACK_TRANSACTION_QUERY.
- Coloana UNDO_SQL furnizeaza codul SQL opus dpdv logic operatiei DML efectuata de tranzactie. Acest cod se poate utiliza pentru a inversa pasii logici efectuati in tranzactie. Sunt cazuri cand acest cod nu este exact opusul tranzactiei originale. Un SQL_UNDO la un DELETE va coespunde cu un INSERT ce nu neaparat va insera o inregistrare la acelasi ROWID de unde a fost sters.

Flashback Transaction Query

- Instructiunea de mai jos va interoga view-ul FLASHBACK_TRANSACTION_QUERY pentru informatii despre tranzactii, incluzand ID-ul tranzactie, operatia, SCN de start si end, utilizatorul operatiei, codul SQL opus logic operatiei:

```
SELECT xid, start_scn, commit_scn, operation,
logon_user, undo_sql
FROM flashback_transaction_query
WHERE xid = HEXTORAW('000200030000002D');
```

```
000200030000002D  195243  195244  DELETE   HR        insert into "HR"."EMP"
("EMPNO","EMPNAME","SALARY") values ('111','Mike','655');
000200030000002D  195243  195244  INSERT   HR        delete from
"HR"."DEPT" where ROWID = 'AAAKD4AABAAAJ3BAAB';
000200030000002D  195243  195244  UPDATE   HR        update "HR"."EMP" set
"SALARY" = '555' where ROWID = 'AAAKD2AABAAAJ29AAA';
000200030000002D  195243  113565  BEGIN  HR
4 rows selected
```

Tabel cu Suport Valid Time

```
-- Crearea tabelei cu Temporal Validity:
CREATE TABLE angajati (
    id NUMBER PRIMARY KEY,
    nume VARCHAR2(100),
    pozitie VARCHAR2(100),
    salariu NUMBER,
    valid_from DATE,
    valid_to DATE,
    PERIOD FOR valid_time (valid_from, valid_to)
);
```

În acest exemplu PERIOD FOR valid_time este folosit pentru a defini perioada de valabilitate a fiecărui rând. valid_from și valid_to reprezintă începutul și sfârșitul perioadei de valabilitate pentru înregistrarea respectivă.

Tabel cu Suport Valid Time

- Inserarea datelor (Create)

```
INSERT INTO angajati (id, nume, pozitie, salariu,
valid_from, valid_to)
VALUES (1, 'Maria Ionescu', 'Manager', 5000, DATE
'2023-01-01', DATE '2024-01-01’);

INSERT INTO angajati (id, nume, pozitie, salariu,
valid_from, valid_to)
VALUES (2, 'Andrei Popescu', 'Developer', 4000, DATE
'2023-05-01', DATE '2024-05-01');
```

Inserează doi angajați, fiecare cu o perioadă de valabilitate specificată.

Tabel cu Suport Valid Time

- Citirea datelor (Read)

Pentru a interoga datele valide la un anumit moment, folosim **AS OF**:

```
SELECT * FROM angajati
  AS OF PERIOD FOR valid_time DATE '2023-06-01’;
```

Returnează toate înregistrările valabile la data de 1 iunie 2023.

- Actualizarea datelor (Update)

Actualizeaza datele unui angajat și a extinde perioada de valabilitate:

```
UPDATE angajati
SET salariu = 5500, valid_to = DATE '2025-01-01'
WHERE id = 1;
```

Actualizează salariul și extinde perioada de valabilitate pentru angajatul cu ID-ul 1 până la data de 1 ianuarie 2025.

Tabel cu Suport Valid Time

- Ștergerea datelor (Delete)

Pentru a șterge o înregistrare specifică din tabelă:

```
DELETE FROM angajati WHERE id = 2;
```

Elimina înregistrarea pentru angajatul cu ID-ul 2.

- Interogare cu Valid Time pentru interval de timp

Interogăm datele valide într-un interval de timp specificat:

```
SELECT * FROM angajati
  WHERE PERIOD FOR valid_time OVERLAPS (DATE '2023-
01-01', DATE '2023-12-31');
```

Returneaza toate înregistrările valide pentru perioada specificată, adică pentru întregul an 2023.

Alte Exemple de Utilizare

- Se poate crea un view ce furnizeaza stari anterioare folosind clauza AS OF in instructiunea SELECT ce defineste view-ul. Daca se specifica o valoare relativa la o anumita instanta de timp, starea anterioara este recalculata la fiecare accesare a view-ului. De exemplu:

```
CREATE VIEW hour_ago AS
  SELECT * FROM angajati
     AS OF PERIOD FOR valid_time TIMESTAMP
(SYSTIMESTAMP - INTERVAL '60' MINUTE);
```

Alte Exemple de Utilizare

- Se poate folosi clauza AS OF in self-joins sau in operatii cum ar fi INTERSECT sau MINUS, pentru a extrage si/sau compara date la timpi diferiti. Rezultatele se pot stoca/salva prin prefixarea Oracle Flashback Query cu:
  - CREATE TABLE AS SELECT
  - INSERT INTO TABLE SELECT
- Interogarea urmatoare reinsereaza in tabela de angajati inregistrarile ce existau acum 1 ora:

```
INSERT INTO angajati
    (SELECT * FROM angajati
     AS OF PERIOD FOR valid_time TIMESTAMP
(SYSTIMESTAMP - INTERVAL '60' MINUTE)
    )
    MINUS SELECT * FROM angajati);
```

Adaugarea Suport VT

- Pentru a adauga unei tabele existente capabilitati VT fara a adauga in mod explicit coloane pentru perioada de timp, la instructiunea ALTER TABLE se poate specifica clauza ADD PERIOD FOR. Daca, de exemplu, la executia CREATE TABLE nu s-au specificat coloanele START_TIME si END_TIME, se poate apela:

```
ALTER TABLE my_emp ADD PERIOD FOR
user_valid_time;
```

- Instructiunea de mai sus adauga doua coloane ascunse tabelei MY_EMP: USER_VALID_TIME_START si USER_VALID_TIME_END. Se pot insera inregistrari ce specifica valori pentru aceste coloane, dar aceste coloane nu apar in rezultatul unei instructiuni DESCRIBE, iar in instructiunea SELECT vor fi afisate si aceste 2 coloane doar daca lista din select include explicit numele acestora.

Oracle References

- http://www.oracle.com/webfolder/technetwork/tutorials/obe/db/12c/r1/ilm/temporal/temporal.html
- https://docs.oracle.com/cd/B28359_01/appdev.111/b28424/adfns_flashback.htm

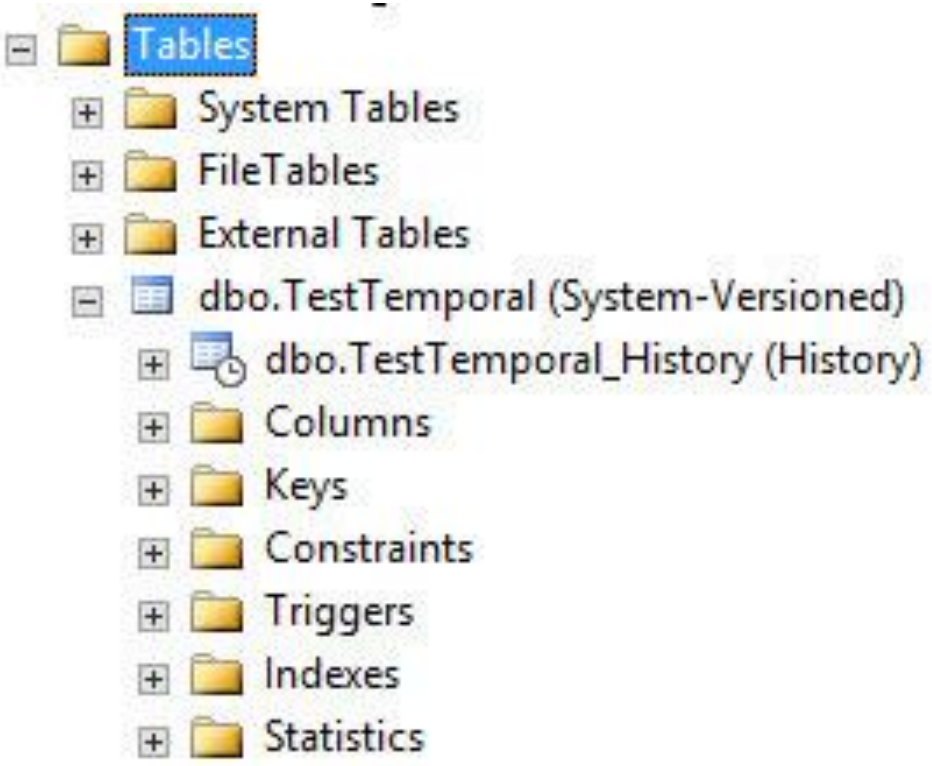
Tabele Temporale in MS Sql Server

Introducere

SQL Server include suport pentru date temporale prin System-Versioned Temporal Tables, introdusă în 2016.

- System-Versioned Temporal Tables: permit urmărirea automată a istoricului modificărilor înregistrărilor. Se folosesc două tabele: una pentru datele curente și alta pentru datele istorice.
- Fiecare înregistrare are două coloane suplimentare, SysStartTime și SysEndTime, care indică perioada în care un rând a fost valabil.
- Interogarea datelor istorice: folosind comanda FOR SYSTEM_TIME, se pot interoga datele într-un anumit interval de timp (AS OF, FROM TO, BETWEEN, etc.).
- SQL Server gestionează automat istoria modificărilor, ceea ce face ca acest mecanism să fie foarte eficient pentru urmărirea datelor pe termen lung.

Introducere



Introducere

- Tabelele temporale (tabele versionate sistem) se mai numesc tabele de asertie, (*assertion* tables), ele capturand durata de viata a unei inregistrari pe baza datelor fizice cand inregistrarile au fost actualizate sau sterse. Pentru moment nu suporta versionare bazata pe date logice.
- Exemplu: se da o tabela ce stocheaza preturi de produse; daca se actualizeaza pretul la 12PM cu o instructiune UPDATE, tabela temporala va inregistra istoricul pretului vechi pana la 12PM in acea zi. Incepand cu 12PM, noul pret este valid. Dar daca s-ar fi dorit o schimbare incepand cu 1PM (schimbare logica)? Ar trebui sincronizat perfect momentul pentru instructiunea de UPDATE la 1PM in loc de 12PM.

Crearea Tabelei Versionate Sistem

- Pentru o tabela temporala nou creata se aplica:
  - Trebuie definita o cheie primara
  - Trebuie definite 2 coloane pentru start si end date cu tipul de data datetime2. Daca este necesar, ele pot fi marcate cu flagul HIDDEN. Aceste coloane vor fi numite coloane pentru perioada SYSTEM_TIME.
  - Nu sunt permisi declansatori (trigger) INSTEAD OF.
  - Nu se poate utiliza In-memory OLTP
- Exista unele limitari:
  - Tabela temporala si cea de istoric nu pot fi FILETABLE
  - Tabela de istoric nu poate avea constrangeri
  - Instructiunile INSERT si UPDATE nu pot referi coloanele perioadei SYSTEM_TIME
  - Datele din tabela de istoric nu pot fi modificate

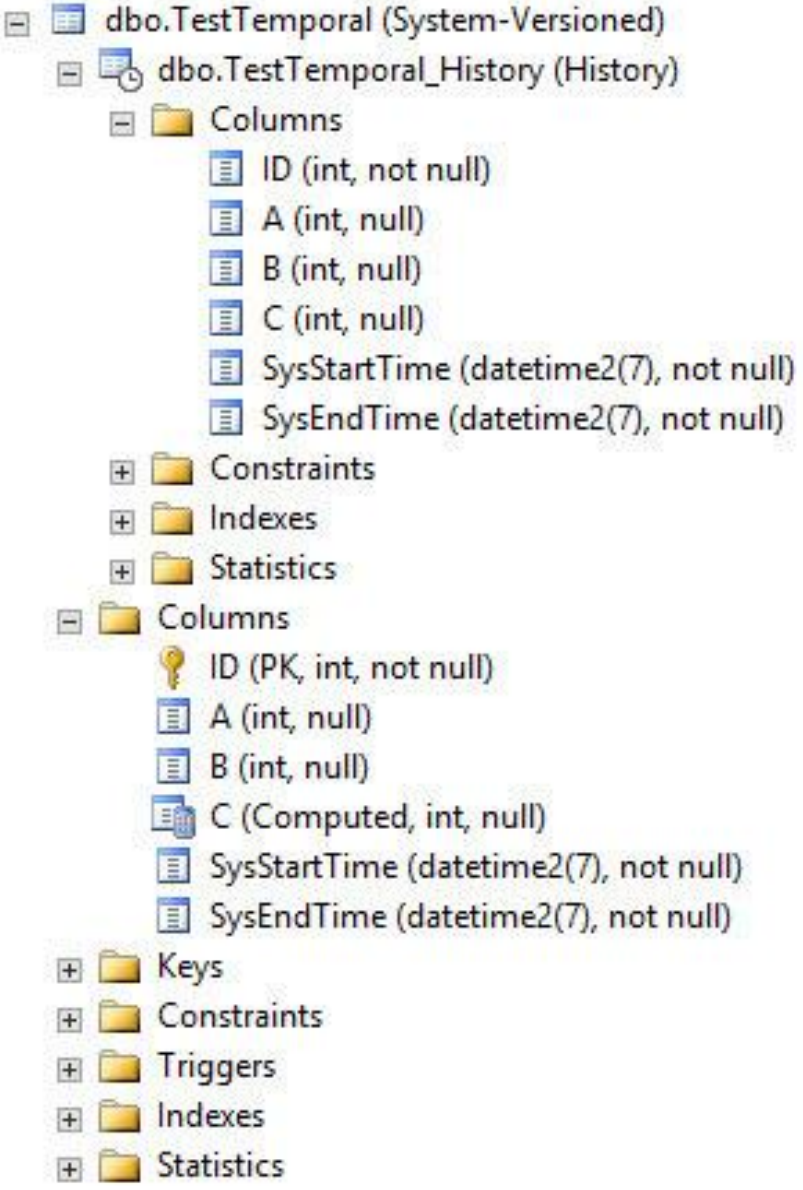
Crearea Tabelei Versionate Sistem

```
CREATE TABLE dbo.TestTemporal
     (ID int primary key
     ,A int
     ,B int
     ,C AS A * B
     ,SysStartTime datetime2 GENERATED ALWAYS AS ROW
START NOT NULL
     ,SysEndTime datetime2 GENERATED ALWAYS AS ROW
END NOT NULL
     ,PERIOD FOR SYSTEM_TIME
(SysStartTime,SysEndTime)) WITH(SYSTEM_VERSIONING =
ON (HISTORY_TABLE=dbo.TestTemporal_History));
```

- Daca nu se specifica un nume pentru tabela de istoric, SQL Server va genera automat unul cu structura: *dbo.MSSQL_TemporalHistoryFor_xxx*, unde *xxx* este id-ul obiect al tabelei principale.

Crearea Tabelei Versionate Sistem

- Tabela de istoric are o multime identica de coloane cu cea de baza, dar fara constrangeri. Are multimea proprie de indexuri si statistici. Crearea unor indexuri clustered poate creste performanta simtitor.



Operatii cu Tabele Versionate Sistem

```
-- Initial Load
INSERT INTO dbo.TestTemporal(ID, A, B, C)
VALUES      (1,2,3,6)
      ,(2,4,5,20)
      ,(3,0,1,0);

SELECT * FROM dbo.TestTemporal;
```

Results | Messages

	ID	A	B	C	SysStartTime	SysEndTime
1	1	2	3	6	2015-07-07 06:20:35.3951761	9999-12-31 23:59:59.9999999
2	2	4	5	20	2015-07-07 06:20:35.3951761	9999-12-31 23:59:59.9999999
3	3	0	1	0	2015-07-07 06:20:35.3951761	9999-12-31 23:59:59.9999999

Operatii cu Tabele Versionate Sistem

```
-- Modify Data
DELETE FROM dbo.TestTemporal
WHERE ID = 2;

UPDATE dbo.TestTemporal
SET A = 5
WHERE ID = 3;
```

```
SELECT * FROM dbo.TestTemporal;
```

100 %

Results | Messages

	ID	A	B	C	SysStartTime	SysEndTime
1	1	2	3	6	2015-07-07 06:20:35.3951761	9999-12-31 23:59:59.9999999
2	3	5	1	5	2015-07-07 06:23:27.7719520	9999-12-31 23:59:59.9999999

```
SELECT * FROM [dbo].[TestTemporal_History];
```

100 %

Results | Messages

	ID	A	B	C	SysStartTime	SysEndTime
1	2	4	5	20	2015-07-07 06:20:35.3951761	2015-07-07 06:23:27.7719520
2	3	0	1	0	2015-07-07 06:20:35.3951761	2015-07-07 06:23:27.7719520

Schimbare Tabela - Versionare Sistem

- SQL Server permite conversia unei tabele existente la o tabela temporala. Pentru aceasta operatie este necesara existenta unei chei primare (daca nu exista se creaza). Se adauga tabelei 2 coloane de tip datetime2 si optiunea GENERATED ALWAYS AS ROW START/END se aplica cu PERIOD FOR SYSTEM_TIME (nameFrom, nameTo).
- Ambele optiuni se executa prin intermediul comenzii ALTER TABLE. A doua comanda ALTER stabileste proprietatea SYSTEM_VERSIONING, furnizand optional un nume pentru proprietatea HISTORY_TABLE.

Schimbare Tabela - Versionare Sistem

```
CREATE TABLE Department_Exist (
      DeptID int NOT NULL PRIMARY KEY CLUSTERED,
      DeptName varchar(50) NOT NULL,
      ManagerID INT  NULL,
      ParentDeptID int NULL )
GO
ALTER TABLE dbo.Department_Exist
ADD SysStartTime datetime2 GENERATED ALWAYS AS ROW START
CONSTRAINT DF_Department_Exist_SysStartTime DEFAULT
SYSUTCDATETIME() NOT NULL,
SysEndTime datetime2 GENERATED ALWAYS AS ROW END
CONSTRAINT DF_Department_Exist_SysEndTime DEFAULT CONVERT
(DATETIME2, '9999-12-31 23:59:59') NOT NULL,
PERIOD FOR SYSTEM_TIME (SysStartTime,SysEndTime)
GO
ALTER TABLE dbo.Department_Exist
      SET (SYSTEM_VERSIONING = ON (HISTORY_TABLE =
dbo.Department_ExistHistory))
GO
```

Resources

https://docs.microsoft.com/en-us/sql/relational-databases/tables/temporal-tables?view=sql-server-ver15
https://www.sqlshack.com/performing-a-sql-server-audit-using-system-versioned-temporal-tables/

PostgreSQL

PostgreSQL nu are suport nativ pentru tabele temporale la nivelul SQL Server sau Oracle, dar oferă posibilitatea implementării acestora folosind extensii și anumite tehnici la nivel de aplicație.

- **Extensia Temporal Tables**: extensie care poate fi adăugată pentru a implementa gestionarea automată a versiunilor istorice ale datelor, similar cu tabelele temporale din SQL Server. Odată activată, permite păstrarea automată a înregistrărilor vechi într-o tabelă de istoric și interogarea acestora folosind funcții specifice.
- Utilizarea manuală a perioadelor de validitate: PostgreSQL permite utilizatorilor să implementeze manual gestionarea datelor temporale, prin adăugarea coloanelor valid_from și valid_to pentru a marca perioadele de validitate ale datelor, împreună cu trigger-uri pentru gestionarea actualizărilor și a ștergerilor.

PostgreSQL

1. Instalarea extensiei temporal_tables

Trebuie instalata și activata extensia pentru a adăuga suport pentru tabele temporale în PostgreSQL.

```
CREATE EXTENSION IF NOT EXISTS temporal_tables;
```

Aceasta instalează extensia necesară pentru gestionarea automată a tabelelor cu versiuni istorice.

PostgreSQL

2. Crearea unei tabele cu suport pentru istoricul datelor

Exemplu de tabela pentru a păstra informații despre angajați:

```
CREATE TABLE angajati (
    id SERIAL PRIMARY KEY,
    nume VARCHAR(100),
    pozitie VARCHAR(100),
    salariu INTEGER
);
```

Activarea gestionarii automate a istoricului pentru această tabelă, folosind extensia temporal_tables.

```
SELECT create_history_table('angajati');
```

Creează automat o tabelă de istoric asociată cu tabelul angajati și păstrează toate modificările făcute asupra datelor.

PostgreSQL

3. Inserarea datelor (Create)

Inserăm câteva date în tabelul angajati:

```
INSERT INTO angajati (nume, pozitie, salariu)
VALUES ('Maria Ionescu', 'Manager', 5000),
       ('Andrei Popescu', 'Developer', 4000);
```

4. Actualizarea datelor (Update)

La actualizarea datele în tabelul principal, modificările anterioare sunt automat salvate în tabelul de istoric.

```
UPDATE angajati
SET salariu = 5500
WHERE nume = 'Maria Ionescu';
```

După actualizare valoarea anterioară a salariului pentru Maria Ionescu va fi stocată automat în tabelul de istoric creat anterior.

PostgreSQL

5. Interogarea datelor din istoric

Pentru a interoga datele istorice se foloseste tabelul de istoric:

```
SELECT * FROM angajati_history;
```

Returnează toate modificările istorice ale tabelei angajati, inclusiv versiunile anterioare ale rândurilor.

6. Ștergerea datelor (Delete)

La fel ca în cazul operațiilor de actualizare, ștergerile sunt păstrate în istoricul tabelei:

```
DELETE FROM angajati WHERE nume = 'Andrei Popescu';
```

După ștergerea lui Andrei Popescu, versiunea anterioară a acestuia va fi păstrată în tabelul de istoric angajati_history.

PostgreSQL

7. Vizualizarea unui anumit moment din istoric

Pentru vizualizarea datelor la un anumit moment, se poate folosi o interogare pentru a extrage versiunea din tabelul de istoric:

```
SELECT * FROM angajati_history
WHERE start_time <= '2023-01-01 00:00:00' AND
end_time > '2023-01-01 00:00:00';
```

Permite accesarea stărilor anterioare ale tabelei la un moment specific în timp.

PostgreSQL

https://wiki.postgresql.org/wiki/Temporal_Extensions

https://github.com/arkhipov/temporal_tables

https://github.com/xocolatl/periods